Universidad de las Fuerzas Armadas ESPE

Departamento de Ciencias de la Computación

Ingeniería en Software

Desarrollo de Aplicaciones Móviles

INFORME DE LABORATORIO 1.2

Desarrollo de una aplicación móvil con principios de diseño

Autores: Julio Blacio, Kerly Chuqui y Josue Guayasamin

NRC: 36872

Docente

Doris Karina Chicaiza Angamarca

Fecha: 2 de noviembre de 2026

Índice de contenidos

Índice de figuras **4**

Índice de tablas **5**

1 Introducción **6**

2 Objetivos **6**

Objetivo general 6

Objetivos específicos 6

3 Trabajos relacionados **7**

Evaluación de enfoques multiplataforma 7

Comunicación entre el framework y Android 7

Rendimiento de tareas de interfaz en Flutter 7

Cálculos y operaciones con datos 8

Adopción y experiencia de desarrollo 8

4 Desarrollo **9**

Conceptos básicos 9

Flutter y composición de widgets 9

Patrón Modelo Vista Controlador 9

Atomic Design 9

Tema y consistencia visual 10

Formularios y navegación 10

Pruebas y precisión monetaria 10

Herramientas 11

Android Studio 11

Flutter y Dart 11

Registro del entorno 11
Desarrollo paso a paso 12
Requisitos de la aplicación 12
Paso 1 Organización del proyecto 13
Paso 2 Creación del proyecto y del tema 15
Paso 3 Implementación del inicio de sesión 16
Paso 4 Implementación de la nueva recarga 18
Paso 5 Cálculo de la comisión y redondeo 19
Paso 6 Comprobante y navegación por rutas 21
Pruebas 22
Estrategia y ejecución 22
Casos y criterios de aceptación 23
Recorrido en Android 24
Resultados y análisis 24
Registro de incidencias 25
Alcance de la verificación 25

5 Conclusiones **26**

6 Recomendaciones **26**

7 Bibliografía **27**

Índice de figuras

Figura 1. Estructura de carpetas del aplicativo 14
Figura 2. Relación entre las responsabilidades MVC 14
Figura 3. Pantalla de inicio de sesión 16
Figura 4. Validación de campos vacíos en el acceso 17
Figura 5. Respuesta ante credenciales incorrectas 18
Figura 6. Formulario principal de nueva recarga 19
Figura 7. Cotización de una recarga de diez dólares con Claro 21
Figura 8. Comprobante de la operación simulada 22

Índice de tablas

Tabla 1. Entorno utilizado para el desarrollo y la verificación 12

Tabla 2. Reglas funcionales de Recargas ESPE 13

Tabla 3. Aplicación de Atomic Design en los componentes 15

Tabla 4. Resultados esperados para una recarga de diez dólares 20

Tabla 5. Resumen de las comprobaciones funcionales 23

Tabla 6. Correspondencia entre requisitos y evidencias 25

1. Introducción

El presente informe documenta el desarrollo de *Recargas ESPE*, un aplicativo Flutter que simula la venta de recargas electrónicas mediante tres pantallas funcionales: inicio de sesión, nueva recarga y comprobante. La práctica aplica el patrón Modelo Vista Controlador (MVC), la composición de interfaces con Atomic Design, un tema visual propio y la navegación mediante rutas. El resultado es una aplicación local que valida los datos, calcula una comisión diferente por operadora y presenta el total de la operación.

La solución se ajustó a la opción 2 del Word de la actividad. El acceso utiliza credenciales locales y las tarifas se definieron para el ejercicio. No se consumen servicios de operadoras, no se procesa dinero y no se almacenan operaciones después de cerrar el proceso. El alcance permite evaluar organización del código, controles Flutter, validación y lógica de programación sin introducir infraestructura ajena a la práctica.

2. Objetivos

Objetivo general

Desarrollar una aplicación móvil de simulación de recargas electrónicas mediante Flutter, MVC y Atomic Design, con tema propio, rutas y validación de entradas, para organizar la lógica de negocio y generar comprobantes verificables de las operaciones.

Objetivos específicos

1. Organizar el proyecto mediante MVC y Atomic Design.
2. Implementar un inicio de sesión con validación local, visibilidad de contraseña, limpieza de campos y mensajes de error.
3. Capturar teléfono, operadora, valor de recarga y forma de pago mediante controles adecuados.
4. Calcular comisiones diferenciadas y el total a pagar, con redondeo explícito a centavos.
5. Presentar un comprobante y permitir una nueva operación o el cierre de sesión.
6. Verificar el funcionamiento mediante pruebas y evidencias de ejecución en Android.

3. Trabajos relacionados

Se revisaron cinco investigaciones sobre desarrollo móvil multiplataforma para relacionar sus aportes con la organización, la implementación y la verificación de Recargas ESPE. Los antecedentes se emplean como contexto; sus resultados experimentales no se presentan como mediciones del aplicativo desarrollado.

Evaluación de enfoques multiplataforma

Rieger y Majchrzak (2019) propusieron un marco de evaluación con criterios agrupados en infraestructura, desarrollo, aplicación y uso. La propuesta permite asignar ponderaciones según las necesidades de un proyecto, en lugar de establecer una clasificación universal de tecnologías. Este trabajo aporta una forma de valorar la solución más allá de su apariencia. En Recargas ESPE se relaciona con la revisión de la organización del código, las herramientas, el funcionamiento de los formularios y la facilidad para comprobar sus reglas; no se aplicó la puntuación ponderada completa del estudio.

Comunicación entre el framework y Android

Biørn-Hansen et al. (2020) compararon el costo de comunicación con las funciones del sistema en aplicaciones Android creadas mediante cinco frameworks multiplataforma y una implementación nativa de referencia. Los resultados variaron según la tecnología y la métrica: algunas alternativas se aproximaron al comportamiento nativo en determinados casos. Su aporte para esta práctica consiste en delimitar las conclusiones de la ejecución en Android. Que Recargas ESPE compile y responda a sus controles demuestra funcionamiento en el entorno probado, pero no permite afirmar equivalencia de rendimiento con una aplicación nativa.

Rendimiento de tareas de interfaz en Flutter

Białkowski y Smołka (2022) desarrollaron tres pares de aplicaciones funcionalmente equivalentes con Android nativo y Flutter. Evaluaron tareas de interfaz en un Huawei P30

mediante Perfetto y observaron mejores tiempos en las implementaciones nativas dentro de esos escenarios. El antecedente permite distinguir una prueba funcional de una medición temporal. En Recargas ESPE se comprobó que los campos, selectores y rutas responden correctamente; para comparar su eficiencia sería necesario definir tareas, instrumentos, repeticiones y dispositivos, como parte de un experimento adicional.

Cálculos y operaciones con datos

Uciński y Dzieńkowski (2022) compararon Flutter y Xamarin mediante aplicaciones equivalentes en Android y Windows. Los escenarios incluyeron cálculos, ordenamiento y operaciones con bases de datos, repetidos para analizar tiempos medios. Los resultados no señalaron un framework superior en todas las tareas. Este estudio se relaciona con la necesidad de comprobar por separado la lógica de una aplicación. En Recargas ESPE se verificaron formatos, límites, comisiones y redondeo mediante pruebas unitarias; no se implementó una base de datos ni se trasladaron al aplicativo los resultados temporales de esa investigación.

Adopción y experiencia de desarrollo

Jošt y Taneski (2025) analizaron tendencias de Flutter, React Native y MAUI utilizando encuestas, actividad de comunidades y datos de demanda laboral. El estudio encontró diferencias entre indicadores de satisfacción y de oportunidades de empleo, de modo que una mayor valoración en un indicador no implica liderazgo en todos los demás. Su relación con esta práctica está en contextualizar Flutter como herramienta con un ecosistema de desarrollo y documentación. Estos indicadores complementan la comprensión del framework, pero no sustituyen la verificación del código, las entradas y la navegación de Recargas ESPE.

4. Desarrollo

Conceptos básicos

Flutter y composición de widgets

Flutter construye interfaces mediante widgets que describen su apariencia y comportamiento. La composición permite formar componentes mayores a partir de controles más pequeños. Los widgets con estado mantienen información que puede cambiar durante la interacción; al actualizar ese estado, la interfaz se reconstruye para reflejarlo (Flutter, s. f.-d). En esta aplicación, la visibilidad de la contraseña y las selecciones del formulario pertenecen al estado de presentación.

Patrón Modelo Vista Controlador

MVC separa los datos y reglas del dominio, la representación visual y la respuesta a las acciones del usuario (MDN Web Docs, s. f.). En el proyecto, el modelo representa operadoras, importes y comprobantes; los controladores validan y coordinan operaciones; las vistas presentan formularios y resultados. La separación permite revisar y modificar cada responsabilidad sin concentrar toda la lógica en los formularios.

La separación se mantuvo al impedir que los controladores importen componentes de interfaz. La decisión de cambiar de pantalla pertenece a la composición de la aplicación, mientras que el controlador devuelve el resultado de la autenticación o el comprobante. Esto permite comprobar la lógica sin iniciar un emulador.

Atomic Design

Frost (s. f.) describe cinco niveles relacionados: átomos, moléculas, organismos, plantillas y páginas. La actividad solicita átomos, moléculas, organismos y pantallas; se aplicaron esos cuatro niveles y un organismo compartido para la disposición general. Un campo de texto y un botón constituyen elementos básicos; una entrada con etiqueta, icono y validación constituye una molécula; el formulario integra varias moléculas; la pantalla compone el conjunto y sus acciones.

MVC y Atomic Design atienden aspectos distintos. El primero organiza responsabilidades funcionales; el segundo organiza la composición visual. Por ello, las carpetas de Atomic Design se encuentran dentro de `vista`, sin desplazar al modelo ni al controlador.

Tema y consistencia visual

`ThemeData` permite compartir colores, tipografía y estilos de componentes entre pantallas (Flutter, s. f.-g). Se definió una paleta verde, un fondo claro, tipografía Roboto y estilos globales para botones y campos. El tema evita repetir decoraciones en cada formulario y conserva una apariencia consistente durante el flujo.

Formularios y navegación

Flutter permite agrupar controles en un `Form`, acceder a su estado mediante una clave y validar sus campos antes de procesar una acción (Flutter, s. f.-b). Los selectores `DropdownButtonFormField` participan en esa validación y evitan escribir libremente opciones fuera de las disponibles (Flutter, s. f.-c).

Las rutas identifican los destinos de navegación. Para esta práctica se definieron rutas nominales y se resolvieron con `onGenerateRoute`. La documentación actual advierte que las rutas nominales tienen limitaciones para aplicaciones con requisitos de navegación más amplios (Flutter, s. f.-e). Aquí se utilizan por el requisito académico y por tratarse de un flujo local de tres pantallas.

Pruebas y precisión monetaria

Las pruebas unitarias verifican funciones o clases; las pruebas de widgets verifican componentes y su interacción; una ejecución en dispositivo permite revisar la aplicación en su plataforma de destino (Flutter, s. f.-f). Estas comprobaciones se complementan: una prueba de cálculo no acredita por sí sola la legibilidad de un formulario.

Se representaron los importes con enteros en centavos, utilizando el tipo `int` de Dart

(Dart, s. f.). El porcentaje de comisión se expresó en puntos base para efectuar la multiplicación con enteros. Esta decisión evita introducir fracciones binarias en el cálculo monetario y hace reproducible la regla de redondeo.

Herramientas

Android Studio

Android Studio ofrece integración con Flutter mediante los complementos de Flutter y Dart. Permite abrir el proyecto, editar archivos y seleccionar un dispositivo de ejecución (Flutter, s. f.-a). Las comprobaciones registradas en esta práctica se realizaron mediante los comandos de Flutter y el emulador Android.

Flutter y Dart

El SDK de Flutter proporciona los comandos de creación, análisis, pruebas y compilación del aplicativo. Dart se utiliza para implementar los modelos, los controladores y los widgets. El SDK de Android y el emulador permiten ejecutar el resultado en la plataforma móvil.

Registro del entorno

La Tabla 1 resume las herramientas comprobadas durante la ejecución de esta práctica. Las versiones del informe 1.1 no se reutilizaron como si fueran actuales.

Tabla 1
Entorno utilizado para el desarrollo y la verificación

Elemento	Registro
Sistema anfitrión	Windows
Flutter	3.47.7, canal estable
Dart	3.13.5
SDK Flutter	`C:\Users\jcbla\flutter\flutter`
SDK Android	`C:\Users\jcbla\AppData\Local\Android\Sdk`
Dispositivo de prueba	AVD Pixel_8_Pro; `emulator-5554`
Sistema del emulador	Android 17, API 37
Dependencias de aplicación	SDK Flutter; sin paquetes externos de ejecución
Pruebas y análisis	`flutter_test`, `integration_test` y `flutter_lints`
Fecha de verificación	2 de noviembre de 2026

Nota. Las versiones corresponden al entorno utilizado para desarrollar y verificar el aplicativo.

Desarrollo paso a paso

Requisitos de la aplicación

El documento de actividad estableció tres pantallas, acceso local, MVC, Atomic Design, tema, rutas y validación. La pantalla de nueva recarga cumple también la función de pantalla principal. No se agregó un menú independiente, para mantener el límite de tres pantallas.

Tabla 2
Reglas funcionales de Recargas ESPE

Dato o comportamiento	Regla aplicada
Acceso de práctica	Usuario `admin`; contraseña `Admin123*`
Teléfono	Diez dígitos con prefijo 09
Valor	USD 1,00 a USD 100,00, inclusive
Formato del valor	Hasta dos decimales; coma o punto como separador
Operadoras	Claro, Movistar y CNT
Comisión simulada	Claro 3 %; Movistar 2,5 %; CNT 2 %
Forma de pago	Efectivo o transferencia, solo como dato de simulación
Redondeo	Centavo más próximo; mitad hacia arriba
Persistencia	Sesión y comprobante únicamente en memoria

Nota. El rango de valores, el formato del teléfono y los porcentajes son decisiones del ejercicio. Las comisiones no representan tarifas oficiales de las operadoras.

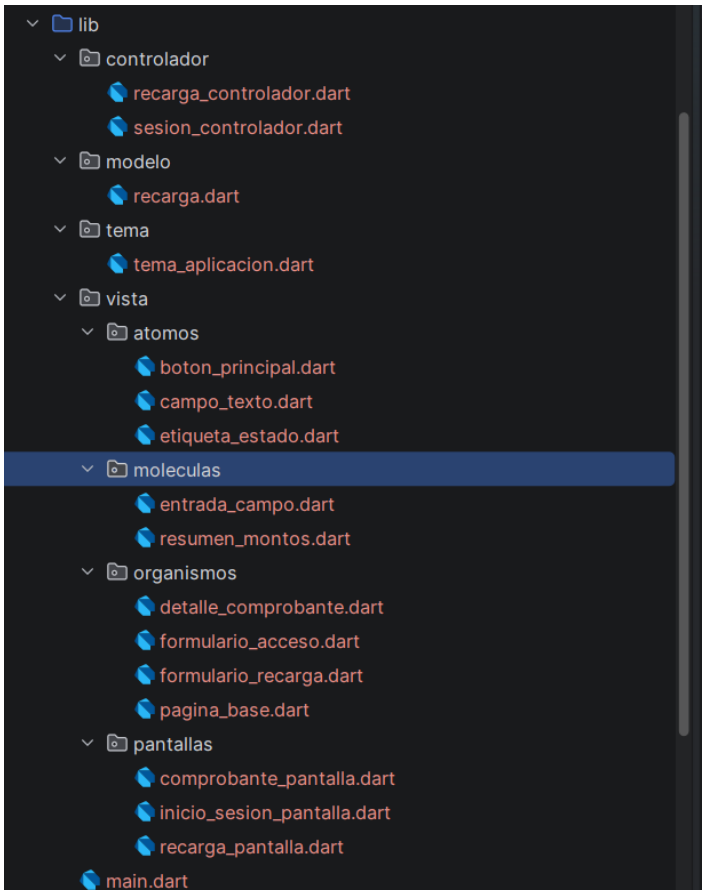
Paso 1 Organización del proyecto

La estructura del aplicativo separa el modelo, los controladores, el tema y las vistas.

Dentro de las vistas se organizan los componentes de Atomic Design.

Figura 1

Estructura de carpetas del aplicativo

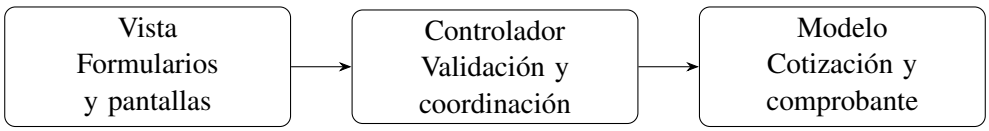


Nota. La captura muestra la organización del modelo, los controladores, el tema y los componentes de la vista.

`SesionControlador` valida las credenciales y mantiene el usuario de la sesión. `RecargaControlador` valida los datos de recarga, produce una cotización y genera el comprobante. En el modelo, `Operadora` define las tasas simuladas, `FormaPago` limita las alternativas y `Comprobante` conserva una instantánea inmutable de la operación.

Figura 2

Relación entre las responsabilidades MVC



Nota. Diagrama elaborado a partir de la organización del código. Las vistas reciben los resultados del controlador y los presentan; el modelo no importa widgets.

Tabla 3
Aplicación de Atomic Design en los componentes

Nivel	Componentes	Responsabilidad
Átomos	CampoTexto, BotonPrincipal, EtiquetaEstado	Entrada, acción y distintivo básicos
Moléculas	EntradaCampo, ResumenMontos	Entrada con contexto y resumen monetario
Organismos	FormularioAcceso, FormularioRecarga, DetalleComprobante, PaginaBase	Bloques completos de interfaz
Pantallas	InicioSesionPantalla, RecargaPantalla, ComprobantePantalla	Composición y acciones de cada destino

Nota. `EntradaCampo` utiliza el átomo `CampoTexto`; los formularios reutilizan las moléculas y los botones.

Paso 2 Creación del proyecto y del tema

Se creó el proyecto con soporte Android y web y se mantuvieron únicamente las dependencias de ejecución del SDK. La plataforma Android se utilizó para la ejecución documentada; la carpeta web queda disponible como destino del proyecto, sin atribuirle una prueba Android.

```
flutter create --platforms=android,web --org ec.edu.espe --project-name
   recargas_espe --empty recargas_espe
```

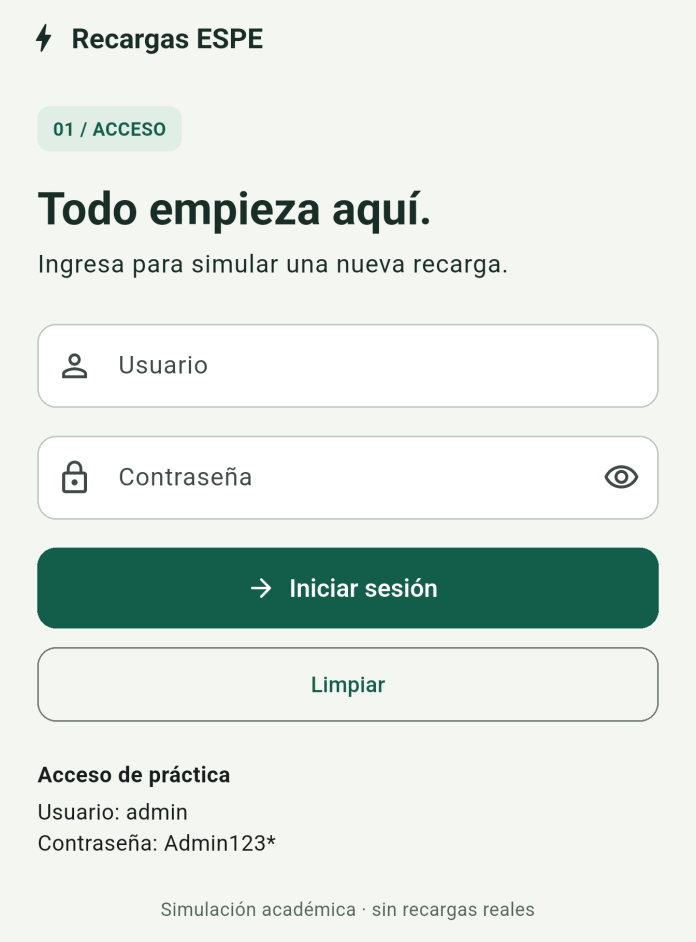
El archivo `tema/tema_aplicacion.dart` concentra el tema claro. Se utilizaron verde `#135E4B`, fondo `#F4F6F2` y texto `#182D26`. Los botones principales mantienen un tamaño mínimo de 52 unidades lógicas y los campos comparten bordes, relleno y espacio interno. `PaginaBase` aporta desplazamiento vertical y limita el ancho del contenido en superficies grandes.

Paso 3 Implementación del inicio de sesión

La primera pantalla incluye usuario, contraseña, control para mostrar u ocultar su contenido, botón Iniciar sesión y botón Limpiar. La validación distingue dos casos: campos vacíos, que muestran un mensaje asociado al campo, y credenciales incorrectas, que generan un mensaje general sin navegar. Al ingresar las credenciales válidas se abre la pantalla de recarga.

El botón Limpiar elimina los textos, los errores y la visibilidad temporal de la contraseña. Las credenciales se muestran como acceso de práctica para facilitar la evaluación. Al estar definidas dentro del código, este mecanismo sirve al ejercicio y no constituye autenticación apta para un servicio real.

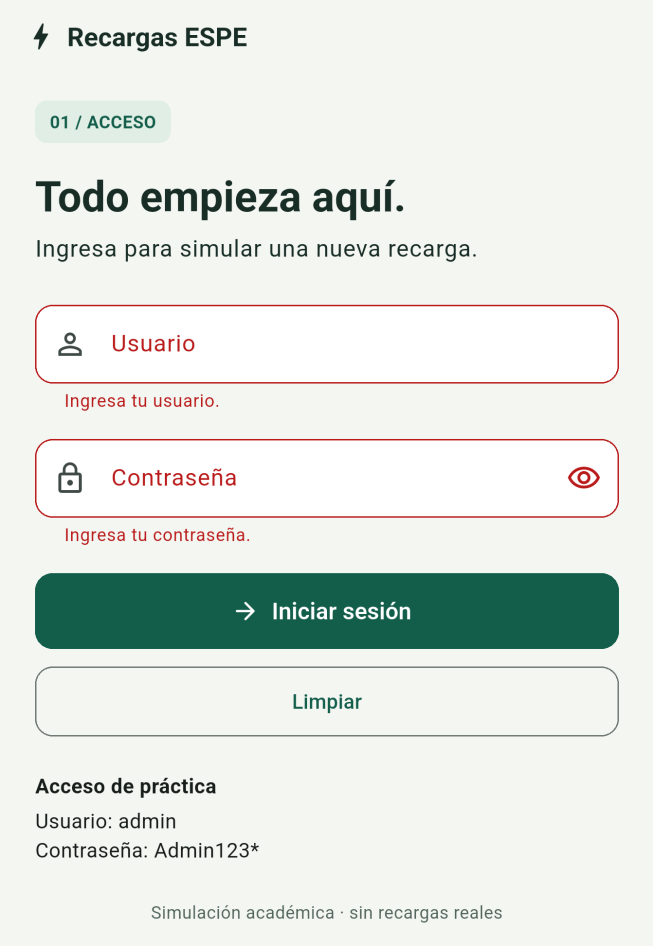
Figura 3
Pantalla de inicio de sesión



Nota. Captura de la aplicación ejecutada en el emulador Android. Se observan los dos campos, el control de visibilidad y las acciones de acceso y limpieza.

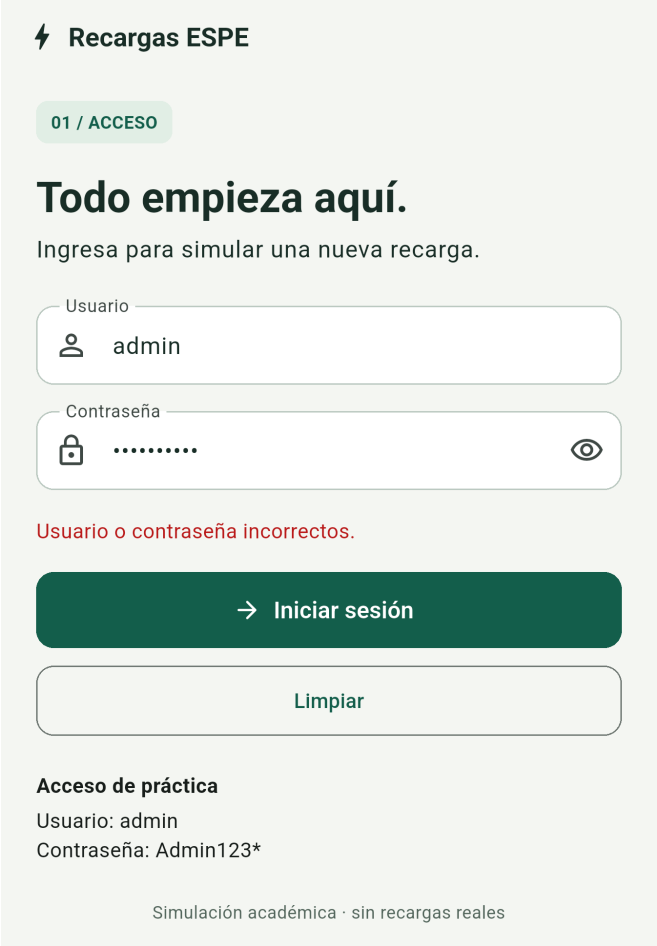
Figura 4

Validación de campos vacíos en el acceso



Nota. Captura de la aplicación ejecutada en el emulador Android. La solicitud de acceso sin datos mantuvo la pantalla y mostró los mensajes correspondientes.

Figura 5
Respuesta ante credenciales incorrectas



Nota. Captura de la aplicación ejecutada en el emulador Android. El mensaje informa el rechazo del acceso; no se abrió la pantalla principal.

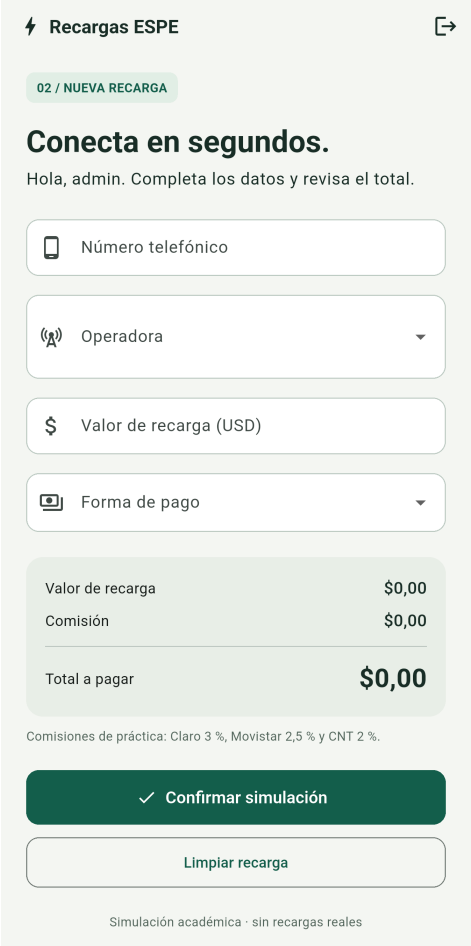
Paso 4 Implementación de la nueva recarga

La segunda pantalla solicita número telefónico, operadora, valor en dólares y forma de pago. El teléfono se captura con teclado telefónico y el monto con teclado numérico decimal; ambos se validan nuevamente mediante las reglas del controlador. Los selectores limitan la operadora y el pago a las opciones definidas.

Al modificar el monto o la operadora se actualiza la cotización visible, sin crear todavía un comprobante. La confirmación solo continúa cuando todo el formulario es válido. Limpiar recarga restablece texto, selecciones y resultado. El resumen presenta valor, comisión y total, para

que la persona pueda revisar el cálculo antes de continuar.

Figura 6
Formulario principal de nueva recarga



Nota. Captura de la aplicación ejecutada en el emulador Android. Las selecciones comienzan vacías y el resumen inicial muestra valores de cero.

Paso 5 Cálculo de la comisión y redondeo

Para una recarga de valor V y una tasa de comisión p, la lógica corresponde a:

$$C = V \times p, \qquad T = V + C.$$

En la implementación, v representa el valor en centavos y b la tasa en puntos base. La comisión entera se calcula con:

$$c = \left\lfloor \frac{v \times b + 5000}{10000} \right\rfloor, \qquad t = v + c.$$

La adición de 5000 aplica el redondeo de mitad hacia arriba para importes positivos. Por ejemplo, USD 1,00 con Movistar produce USD 0,025 de comisión antes de redondear; el comprobante registra USD 0,03 y un total de USD 1,03. No se redondea el monto de entrada: si contiene más de dos decimales, se rechaza.

Tabla 4

Resultados esperados para una recarga de diez dólares

Operadora	Comisión	Valor de comisión	Total a pagar
Claro	3 %	USD 0,30	USD 10,30
Movistar	2,5 %	USD 0,25	USD 10,25
CNT	2 %	USD 0,20	USD 10,20

Nota. Los tres casos se verificaron con pruebas unitarias. La Figura 7 documenta el caso Claro en la interfaz Android.

Figura 7
Cotización de una recarga de diez dólares con Claro



Nota. Captura de la aplicación ejecutada en el emulador Android. La interfaz presenta USD 0,30 de comisión y USD 10,30 de total antes de confirmar.

Paso 6 Comprobante y navegación por rutas

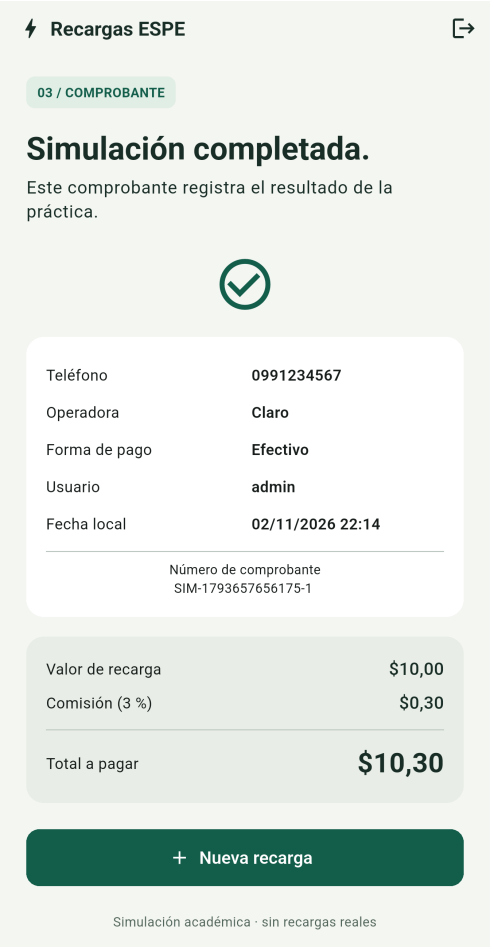
La tercera pantalla presenta teléfono, operadora, pago, usuario, fecha local, identificador, valor, comisión y total. El código se forma con una marca temporal y una secuencia en memoria; distingue operaciones durante la sesión del proceso, sin afirmar unicidad entre instalaciones. El texto indica que la operación es una simulación.

En `main.dart` se definieron las rutas `/`, `/recarga` y `/comprobante`. El generador de rutas comprueba la sesión antes de presentar pantallas internas. El destino del comprobante requiere un resultado disponible; si no existe, se presenta el formulario. La acción Nueva recarga

elimina el comprobante anterior y abre un formulario nuevo. Cerrar sesión elimina usuario y resultado, y reemplaza la pila de navegación con el acceso.

Figura 8

Comprobante de la operación simulada



Nota. Captura de la aplicación ejecutada en el emulador Android. El resultado conserva los datos confirmados y permite comenzar una nueva recarga. El teléfono se utilizó únicamente como dato de prueba y no fue contactado.

Pruebas

Estrategia y ejecución

Se ejecutaron pruebas unitarias de controladores y modelos, y pruebas de widgets que interactúan con la aplicación. La primera prueba de acceso falló sobre el proyecto vacío, porque todavía no existían sus controles. Después de implementarlos se verificó el comportamiento

funcional, sin sustituirlo por capturas de una maqueta.

La suite final comprende 30 pruebas unitarias y siete pruebas de widgets, para un total de 37. Las primeras comprueban credenciales, formatos, límites, tasas, redondeo, creación y limpieza del comprobante. Las segundas verifican controles obligatorios, mensajes, visibilidad de contraseña, limpieza, protección de ruta, flujo completo y adaptación a pantalla estrecha.

```
flutter analyze

flutter test --reporter expanded

flutter build apk --debug
```

Casos y criterios de aceptación

Tabla 5
Resumen de las comprobaciones funcionales

Caso	Entrada o acción	Criterio comprobado
P01	Acceso vacío	Mensajes en usuario y contraseña; sin navegación
P02	Contraseña incorrecta	Rechazo del acceso
P03	Credenciales de práctica	Apertura del formulario principal
P04	Mostrar y limpiar contraseña	Cambio de visibilidad y reinicio de campos
P05	Teléfono incompleto o inválido	Rechazo por formato
P06	Monto negativo, texto o más de dos decimales	Rechazo de la entrada
P07	Montos 1,00 y 100,00	Aceptación de ambos límites
P08	Montos 0,99 y 100,01	Rechazo fuera del rango
P09	Diez dólares por operadora	Totales 10,30; 10,25 y 10,20
P10	Un dólar con Movistar	Comisión 0,03 tras redondear
P11	Confirmar sin selecciones	Permanencia en el formulario y errores
P12	Recarga completa y Nueva recarga	Comprobante correcto y formulario limpio
P13	Ruta interna sin sesión	Presentación del acceso
P14	Pantalla de 320 por 640 y texto al 150 %	Desplazamiento sin desbordamientos detectados

Nota. La tabla agrupa comportamientos; no equivale al número de funciones de prueba. Los resultados se verificaron mediante la suite automatizada.

Recorrido en Android

Se compiló el APK de depuración y se ejecutó en el emulador indicado en la Tabla 1. Una prueba de integración Android adicional recorrió acceso vacío, credenciales incorrectas, acceso válido, formulario de recarga, comprobante, nueva operación y cierre de sesión. Las Figuras 3 a 8 se capturaron directamente desde la interfaz ejecutada mediante `integration_test` y se conservaron como archivos PNG. Esta prueba de integración es adicional a las 37 pruebas unitarias y de widgets.

```
flutter drive --driver=test_driver/integration_test.dart --target=
   integration_test/recorrido_test.dart -d emulator-5554
```

La captura de la interfaz acredita el estado visible en ese momento. Las comprobaciones del cálculo para otras operadoras y de entradas límite se respaldan con las pruebas automatizadas, sin presentar una captura de Claro como evidencia visual de las otras tasas.

Resultados y análisis

El análisis estático finalizó sin incidencias y la suite automatizada registró 37 pruebas aprobadas. La compilación Android produjo un APK de depuración. El recorrido en el emulador mostró las tres pantallas y un comprobante de USD 10,30 para una recarga de USD 10,00 con comisión del 3 %.

La estructura de carpetas permite identificar dónde modificar cada comportamiento. Cambiar una tasa afecta al modelo; modificar la validación afecta al controlador; ajustar un campo compartido afecta a sus componentes visuales. La composición evita repetir el formulario de acceso en otra pantalla y reutiliza el resumen monetario en la cotización y el comprobante.

Tabla 6
Correspondencia entre requisitos y evidencias

Requisito	Implementación	Evidencia
Tres pantallas funcionales	Acceso, recarga y comprobante	Figuras 3 a 8
MVC	modelo, controlador y vista	Figura 2 y código
Atomic Design	Cuatro niveles dentro de vista	Tabla 3 y código
Tema propio	tema_aplicacion.dart	Capturas Android
Rutas	main.dart y RutasApp	Prueba del flujo y de acceso
Validación de entradas	Controladores y Form	Tabla 5 y suite de pruebas
Comisiones diferentes	Tasas por Operadora	Tabla 4 y pruebas unitarias
Acceso con controles requeridos	FormularioAcceso	Figuras 3 a 5 y widgets

Nota. La evidencia combina inspección del código, pruebas y ejecución. Cada método acredita una parte del requisito.

Registro de incidencias

Durante la verificación se detectó que el título del encabezado y el selector de pago desbordaban su espacio en una pantalla de 320 unidades de ancho con texto ampliado al 150 %. El título se hizo flexible y se habilitó la expansión del selector dentro del ancho disponible. La prueba se repitió y finalizó sin excepciones de diseño. También se corrigió la consulta de visibilidad de contraseña en la prueba: esa propiedad se comprueba en el `TextField` interno, no en `TextFormField`.

Alcance de la verificación

Los resultados acreditan las reglas e interacciones cubiertas por la suite y el funcionamiento observado en un emulador Android. No constituyen pruebas de carga, una auditoría completa de accesibilidad, mediciones de rendimiento ni validación en todos los

tamaños y dispositivos. La autenticación es local y el comprobante no tiene validez comercial. No se implementaron servidor, base de datos, historial persistente, pagos ni servicios de recarga reales.

5. Conclusiones

1. Se desarrolló una aplicación de recargas con las tres pantallas requeridas y un flujo completo desde el acceso hasta el comprobante. Los datos ingresados se validan antes de confirmar y los mensajes permiten identificar errores sin avanzar a una operación incompleta.
2. La aplicación combinó MVC con Atomic Design. Las carpetas en español conservan el enfoque trabajado en clase y separan las reglas de negocio de los componentes de presentación. El tema compartido mantuvo colores, tipografía y controles consistentes.
3. Las tasas diferenciadas y el cálculo en centavos produjeron resultados reproducibles, incluido el redondeo de medio centavo. Las 37 pruebas aprobadas, el análisis estático sin incidencias y el recorrido Android respaldan el alcance funcional descrito, sujeto a los límites de la simulación.

6. Recomendaciones

1. Mantener las tasas de práctica identificadas como valores simulados. Si la aplicación se conecta posteriormente con una operadora o un sistema de pagos, reemplazar la validación local por autenticación de servidor y verificar las reglas del servicio antes de procesar operaciones.
2. Conservar la separación de responsabilidades al agregar funciones y ejecutar las pruebas cuando cambien límites, tasas o navegación. Ampliar la verificación a dispositivos físicos, más escalas de texto y otras versiones de Android antes de plantear una distribución.
3. Conservar las capturas de las comprobaciones y repetir las pruebas antes de modificar las reglas de negocio. Una distribución futura requeriría configurar y firmar una versión de publicación.

7. Bibliografía

Białkowski, D., & Smołka, J. (2022). Evaluation of Flutter framework time efficiency in context of user interface tasks. *Journal of Computer Sciences Institute, 25*, 309–314. https://doi.org/10.35784/jcsi.3007

Biørn-Hansen, A., Rieger, C., Grønli, T.-M., Majchrzak, T. A., & Ghinea, G. (2020). An empirical investigation of performance overhead in cross-platform mobile development frameworks. *Empirical Software Engineering, 25*(4), 2997–3040. https://doi.org/10.1007/s10664-020-09827-6

Dart. (s. f.). *Built-in types*. Recuperado el 2 de noviembre de 2026, de https://dart.dev/language/built-in-types

Flutter. (s. f.-a). *Android Studio and IntelliJ*. Recuperado el 2 de noviembre de 2026, de https://docs.flutter.dev/tools/android-studio

Flutter. (s. f.-b). *Build a form with validation*. Recuperado el 2 de noviembre de 2026, de https://docs.flutter.dev/cookbook/forms/validation

Flutter. (s. f.-c). *DropdownButtonFormField class*. Recuperado el 2 de noviembre de 2026, de https://api.flutter.dev/flutter/material/DropdownButtonFormField-class.html

Flutter. (s. f.-d). *Flutter architectural overview*. Recuperado el 2 de noviembre de 2026, de https://docs.flutter.dev/resources/architectural-overview

Flutter. (s. f.-e). *Navigate with named routes*. Recuperado el 2 de noviembre de 2026, de https://docs.flutter.dev/cookbook/navigation/named-routes

Flutter. (s. f.-f). *Testing Flutter apps*. Recuperado el 2 de noviembre de 2026, de https://docs.flutter.dev/testing/overview

Flutter. (s. f.-g). *Use themes to share colors and font styles*. Recuperado el 2 de noviembre de 2026, de https://docs.flutter.dev/cookbook/design/themes

Frost, B. (s. f.). Atomic design methodology. En *Atomic design*. https://atomicdesign.bradfrost.com/chapter-2/

Jošt, G., & Taneski, V. (2025). State-of-the-art cross-platform mobile application development frameworks: A comparative study of market and developer trends. *Informatics, 12*(2), Artículo 45. https://doi.org/10.3390/informatics12020045

MDN Web Docs. (s. f.). *MVC*. Recuperado el 2 de noviembre de 2026, de https://developer.mozilla.org/en-US/docs/Glossary/MVC

Rieger, C., & Majchrzak, T. A. (2019). Towards the definitive evaluation framework for cross-platform app development approaches. *Journal of Systems and Software, 153*, 175–199. https://doi.org/10.1016/j.jss.2019.04.001

Uciński, M., & Dzieńkowski, M. (2022). A comparative analysis of performance of Flutter and Xamarin development frameworks. *Journal of Computer Sciences Institute, 25*, 366–370. https://doi.org/10.35784/jcsi.3025